

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配
练习

流编辑器 sed

bash 正则表
达式

Perl 正则表
达式

Python 正则表
达式

调试

常见问题

正则表达式

续本达

清华大学 工程物理系

2025-07-09 清华

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配

练习

流编辑器 sed

bash 正则表

达式

Perl 正则表

达式

Python 正则表

达式

调试

常见问题

`pcre2-utils` Perl 正则表达式工具

`libpcre2-dev` Perl 正则表达式文档和开发工具

`libregex-debugger-perl` 正则表达式调试器

```
# Debian
```

```
apt install pcre2-utils libpcre2-dev libregex-debugger-perl
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配
练习

流编辑器 sed

bash 正则表
达式

Perl 正则表
达式

Python 正则表
达式

调试

常见问题

```
wget "http://hep.tsinghua.edu.cn/~orv/pd/usr_share_doc_list.txt"
```

```
--2022-07-27 11:59:50-- http://hep.tsinghua.edu.cn/~orv/pd/usr_share_doc_list.txt
Resolving hep.tsinghua.edu.cn... 101.6.6.219, 2402:f000:1:416:101:6:6:219
Connecting to hep.tsinghua.edu.cn|101.6.6.219|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 128653 (126K) [text/plain]
Saving to: 'usr_share_doc_list.txt'
```

```
wget "http://hep.tsinghua.edu.cn/~orv/pd/sources.list"
```

```
--2022-07-27 12:21:55-- http://hep.tsinghua.edu.cn/~orv/pd/sources.list
Resolving hep.tsinghua.edu.cn... 101.6.6.219, 2402:f000:1:416:101:6:6:219
Connecting to hep.tsinghua.edu.cn|101.6.6.219|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1784 (1.7K)
Saving to: 'sources.list'
```

```
sources.list          0%[                               ]      0  --.-KB/s
sources.list          100%[=====>]    1.74K  --.-KB/s    in 0s
```

```
2022-07-27 12:21:55 (200 MB/s) - 'sources.list' saved [1784/1784]
```

```
wget "http://hep.tsinghua.edu.cn/~orv/pd/ser.h5"
```

```
--2022-07-27 13:06:55-- http://hep.tsinghua.edu.cn/~orv/pd/ser.h5
Resolving hep.tsinghua.edu.cn... 101.6.6.219, 2402:f000:1:416:101:6:6:219
Connecting to hep.tsinghua.edu.cn|101.6.6.219|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3648 (3.6K)
Saving to: 'ser.h5'
```

```
ser.h5          0%[                               ]      0  --.-KB/s
ser.h5         100%[=====>]    3.56K  --.-KB/s   in 0s
```

```
2022-07-27 13:06:55 (301 MB/s) - 'ser.h5' saved [3648/3648]
```

- 数据流水线构建最佳工具。
 - 程序写得尽可能短，特定场景下 Python 之外的工具更能胜任。
 - 不要一切都用“我人生中掌握的第一门程序语言”实现。
- 函数式编程：一切都是函数，从输入到输出的映射。
 - 数据驱动编程：循环是暗含的，只关注组成单元的处理。
- make 处理的替换阶段与执行阶段。
- guile 的 scheme 语言在 make 中的调用。

```
target: source
    program source target #如何做
    program $^ $@ #如何做
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配

练习

流编辑器 sed

bash 正则表

达式

Perl 正则表

达式

Python 正则表

达式

调试

常见问题

```
seq 50 | grep 7
```

7

17

27

37

47

如何匹配更复杂的规律？

- 正则表达式
- 更高级的文件管理
- 文件内容识别

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配

练习

流编辑器 sed

bash 正则表

达式

Perl 正则表

达式

Python 正则表

达式

调试

常见问题

- 所有以 3 结尾的两位数

```
seq 50 | grep .3
```

13

23

33

43

- 23、233、2333、.....

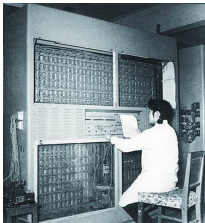
```
seq 10000 | grep -E ^23+$
```

23

233

2333

- “自动机” (Automata) 一词源自希腊语 “αυτόματα”，意思是 “self-acting(自动运行的)”，单数形式是 Automaton。自动机是一种抽象的计算设备，它能够自动按照预定的操作序列运行。



图：1958 年，我国研制成功第一台通用数字电子计算机 103 机

- “形式语言 (formal languages)” 是用精确的数学、机器或规则可定义的语言。与自然语言不同，一个形式语言作为一个集合，需要有某种明确的标准来定义一个字符串是否是它的元素。形式语言可以通过以下方式定义
 - ① 由某种形式文法 (grammar) 生成的字符串；
 - ② 由特定正则表达式 (regular expression) 描述或匹配的字符串；
 - ③ 由某种自动机 (如图灵机或有限状态自动机) 接受的字符串；

- 1921 年希尔伯特 (D. Hilbert) 意识到大多数数学证明都可以在一阶逻辑 (first-order logic) 中形式化, 并且数学的各种概念可以归约为自然数和简单的集合概念。他认为, 如果能够澄清处理自然数及其集合的一阶理论的有限性性质 (特别是无矛盾性), 便可为数学体系奠定坚实的基础。
- 无矛盾性问题最终可以归结为一阶逻辑的判定问题 (Entscheidungsproblem)。因此, Hilbert 在 1928 年明确指出: “判定问题是数理逻辑的核心问题。”
- 哥德尔不完备定理 (1931) 动摇了这一构想: 在适当复杂的公理系统中, 存在不可以在该证明系统内被证明的真命题。这揭示了判定问题在本质上的不可解性, 从根本上限制了形式化数学的边界。

- 多数情况下文本处理的 最佳工具
- 优秀工具的典范：
 - ① 有严格的数学模型 – 用户和开发者之间沟通容易
 - ② 有标准或约定俗成的语法 – 知识迁移
 - ③ 有广泛的实现 – 学好正则表达式，走遍天下都不怕
- 实验物理用户：
 - 只要能写出正确的表达式，就可以使用高性能的正则表达式引擎。

参考书

- Jeffrey Friedl, Mastering Regular Expressions
- Michael Sipser, Introduction to the Theory of Computation

正则表达式的形式定义

设 Σ 是字母表（不可分符号集）， A 和 B 是字母表 Σ 上的有限字符串的集合，我们定义：

并 (Union) $A \cup B = \{x \mid x \in A \text{ 或 } x \in B\}$

连接 (Concatenation) $A.B = \{x.y \mid x \in A \text{ 且 } y \in B\}$

Kleene 星号 (Kleene star) $A^* = \bigcup_{k \geq 0} A^k = \bigcup_{k \geq 0} \{x_1 x_2 \dots x_k \mid i \leq k \rightarrow x_i \in A\}$

例如

$$\{a, ab\}^* = \{\varepsilon, a, ab, aab, aba, aa, abab, aaa, aaab, \dots\}$$

R 是正则表达式 (Regular Expression, RE)，若 R 通过以下归纳定义得到：

- $\{a\}, a \in \Sigma$ ，
- $\{\varepsilon\}$ ，即空字符串（长度为 0）的集合，
- $\emptyset$ （空集）
- 若 R_1, R_2 是 RE，则 $R_1 \cup R_2$ 亦然（对 $\cup$ 运算封闭），
- 若 R_1, R_2 是 RE，则 $R_1.R_2$ 亦然（对 $.$ 运算封闭），
- 若 R_1 是 RE，则 $(R_1)^*$ 亦然（对 $*$ 运算封闭）。

正则表达式的形式定义

设 Σ 是字母表（不可分符号集）， A 和 B 是字母表 Σ 上的有限字符串的集合，我们定义：

并 (Union) $A \cup B = \{x \mid x \in A \text{ 或 } x \in B\}$

连接 (Concatenation) $A.B = \{x.y \mid x \in A \text{ 且 } y \in B\}$

Kleene 星号 (Kleene star) $A^* = \bigcup_{k \geq 0} A^k = \bigcup_{k \geq 0} \{x_1 x_2 \dots x_k \mid i \leq k \rightarrow x_i \in A\}$

例如

$$\{a, ab\}^* = \{\varepsilon, a, ab, aab, aba, aa, abab, aaa, aaab, \dots\}$$

R 是正则表达式 (Regular Expression, RE)，若 R 通过以下归纳定义得到：

- $\{a\}, a \in \Sigma$ ，
- $\{\varepsilon\}$ ，即空字符串（长度为 0）的集合，
- $\emptyset$ （空集）
- 若 R_1, R_2 是 RE，则 $R_1 \cup R_2$ 亦然（对 $\cup$ 运算封闭），
- 若 R_1, R_2 是 RE，则 $R_1.R_2$ 亦然（对 $.$ 运算封闭），
- 若 R_1 是 RE，则 $(R_1)^*$ 亦然（对 $*$ 运算封闭）。

以下命题等价：

- R 是正则表达式
- R 是被有限自动机接受的语言
- R 是被 Chomsky 的 3-型文法生成的语言

有限自动机 $\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$ ，其中 Q 是有限的状态集， Σ 是有限的字母表， $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ 是状态转移关系， $q_0 \in Q$ 是初始状态， $F \subset Q$ 是接受状态的集合。

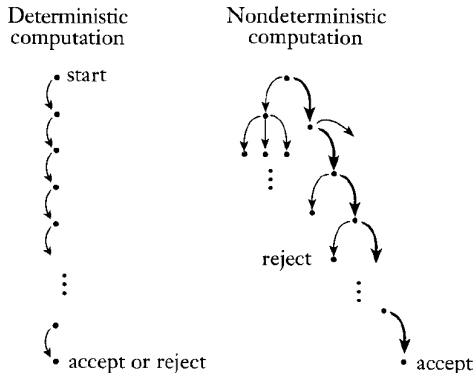
3-型文法满足右线性或左线性

右线性的例子：

$$A \rightarrow wB, A \rightarrow w, A \rightarrow \varepsilon$$

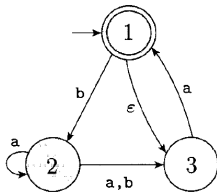
其中 A, B 表示可分符号 (non-terminal symbol)， $w \in \Sigma$ 是不可分符号 (terminal symbol)。

确定与非确定自动机

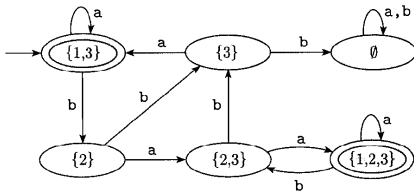


对于任意的非确定型自动机，存在与其等价的确定型自动机。即，任何被非确定型自动机接受的语言，都可以被某一个确定型自动机接受。

- 字母表 $\Sigma = \{a, b\}$ 。非确定型有限自动机 $\mathcal{M}$ ：



- 一个与 $\mathcal{M}$ 等价的确定型自动机：



定义与示例

- $L(M)$ = 被自动机 M 接受的字符串的集合
= 被 M 识别的语言
= 被 M 计算的函数
- 一个语言 L' 是正则的，当且仅当存在一个 DFA M 使得 $L' = L(M)$ 。
- 示例：
 - $L' = \{w \mid w \text{ 以 } 1 \text{ 结尾}\}$ 是正则的。
 - $L' = \{w \mid w \text{ 以 } 1 \text{ 开头}\}$ 是正则的。
 - $L' = \{w \mid w \text{ 有奇数个 } 1\}$ 是正则的。

已知某些语言是正则的

- **正则语言的补集也是正则的。**：若 $A \subseteq \Sigma^*$ 是正则语言，则 $\neg A$ (A 的补集) 也是正则语言，其中 $\neg A = \{w \in \Sigma^* \mid w \notin A\}$ 。
- 为什么这很重要？
- 因为它告诉我们正则语言类在补集操作下是封闭的。封闭性质在形式语言理论中非常基础，允许我们在保持某些性质的同时，从现有语言构造新的语言。
- **关键步骤：**
 - ① 从识别 A 的 DFA M 开始。
 - ② 构造一个新的 DFA M' ，交换终止状态和非终止状态，即 M' 的终止状态为 $Q \setminus F$ 。
 - ③ 证明 M' 现在识别 $\neg A$ ，即 $\neg A$ 是正则的。

- **完整的 DFA 补集构造** 假设 DFA M 是完整的，即对于每个状态和每个字母表中的符号都有定义的转移。如果 M 不完整，我们可能需要添加一个“死状态”来使其完整，然后再取补集。
- **针对 DFA** 对于非确定性自动机 NFA，我们不能直接交换终止状态和非终止状态，因为 NFA 有不同的接受标准。但对于有限输入的自动机而言，任何 NFA 都可以转换为 DFA，结果仍然成立。

正则语言在其他操作下也是封闭的.

- **并集**: 如果 A 和 B 是正则的, 则 $A \cup B$ 是正则的。

要点:

- 设 DFA $M_1 = (Q_1, \Sigma_1, \delta_1, q_0^1, F_1)$ 和 $M_2 = (Q_2, \Sigma_2, \delta_2, q_0^2, F_2)$ 满足 $L(M_1) = A$ 和 $L(M_2) = B$ 。
- 构造新 DFA $M = (Q, \Sigma, \delta, q_0^1, F_1)$:
 - 状态集 $Q = Q_1 \times Q_2$,
 - 转移函数为 $\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$,
 - 接受条件为 $F = \{(q_1, q_2) : q_1 \in F_1 \text{ 或 } q_2 \in F_2\}$.
- 证明 $L(M) = L(M_1) \cup L(M_2)$.

- **交集**: 根据德摩根定律, $A \cap B = \neg(\neg A \cup \neg B)$ 。

语言 L 的反转

$$L^R = \{w_k \cdots w_1 \mid w_1 \cdots w_k \in L\}$$

语法要素：类算术运算的组合规则

- 所有字符默认与自己匹配
- `.` 代表任意字符
- `^`, `$` 开始与结束
- `*` 任意重复
 - `+` 至少一次重复
 - `?` 0 或 1 次
 - `{n,m}` 重复 `n` 至 `m` (包含) 次
- `()` 组合
- `|` 或
- `[]` 字符集
 - `[0-9]`
 - `[a-zA-Z]`
 - `[^abc]`
- `\<`, `\>` 单词的开始和结束处的空白符
- `\b` 单词两边的空白符
- `\B` 非单词两边的空白符, `\b` 的补集

语法要素：类算术运算的组合规则

- 所有字符默认与自己匹配
- `.` 代表任意字符
- `^`, `$` 开始与结束
- `*` 任意重复
 - `+` 至少一次重复
 - `?` 0 或 1 次
 - `{n,m}` 重复 `n` 至 `m` (包含) 次
- `()` 组合
- `|` 或
- `[]` 字符集
 - `[0-9]`
 - `[a-zA-Z]`
 - `[^abc]`
- `\<`, `\>` 单词的开始和结束处的空白符
- `\b` 单词两边的空白符
- `\B` 非单词两边的空白符, `\b` 的补集

语法要素：类算术运算的组合规则

- 所有字符默认与自己匹配
- `.` 代表任意字符
- `^`, `$` 开始与结束
- `*` 任意重复
 - `+` 至少一次重复
 - `?` 0 或 1 次
 - `{n,m}` 重复 `n` 至 `m` (包含) 次
- `()` 组合
- `|` 或
- `[]` 字符集
 - `[0-9]`
 - `[a-zA-Z]`
 - `[^abc]`
- `\<`, `\>` 单词的开始和结束处的空白符
- `\b` 单词两边的空白符
- `\B` 非单词两边的空白符, `\b` 的补集

基础正则表达式 Basic RE

- `? + { | ( )` 是字符本身，需要特殊含义得加上 `\`，例如 `\? \+ 等`

扩展正则表达式 Extended RE

- GNU 环境里与基础正则表达式功能完全一致，只是语法规约定相反，`\? \+ 等`代表“?”和“+”本身，`? + 等有特殊含义`。

Perl 正则表达式 Perl-Compatible RE

- 与 Perl 5 语言正则表达式有同样功能
- 超出有限状态自动机的功能：递归、条件、引用等
 - 严谨来讲已经不再是数学意义上的正则表达式，但更加实用

- 基本功能：检验输入的第一行是否与正则表达式匹配，若是则输出该行。
- `egrep` 等价于 `grep -E`，扩展正则表达式

```
file $(which egrep)
```

```
/bin/egrep: POSIX shell script, ASCII text executable
```

```
cat /bin/egrep
```

```
#!/bin/sh  
exec /bin/grep -E "$@"
```

文档

- `man grep`
- `info grep`
- 源于 `ed` 的语法 `global/regular expression/print` → `g/re/p`

- ^, \$ 开始与结束
- + 至少一次重复
- ? 0 或 1 次
- () 组合

```
seq 10000 | egrep ^2?3+$
```

```
3
23
33
233
333
2333
3333
```

```
seq 10000 | egrep '^ (23)*$'
```

```
23
2323
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配

练习

流编辑器 sed

bash 正则表

达式

Perl 正则表

达式

Python 正则表

达式

调试

常见问题

```
seq 10000 | egrep '^ (23?)+$'
```

2
22
23
222
223
232
2222
2223
2232
2322
2323

- 正则表达式写下了人类语言已经很难描述匹配的规则。

- | 或
- [] [0-9] 字符集

```
seq 10000 | egrep '^ (2(3|4))+ $'
```

```
23
24
2323
2324
2423
2424
```

```
seq 10000 | egrep '^ (23[3-6])+ $'
```

```
233
234
235
236
```

- $\{n,m\}$ 重复 n 至 m (包含) 次

```
seq 1000000 | egrep '^23{2,4}$'
```

233

2333

23333

- 在 [] 之内有效
 - [:alpha:] 字母
 - [:digit:] 数字
 - [:alnum:] 字母或数字
 - \w 与 [[[:alnum:]]] 同义, \W 与 [^[:alnum:]] 同义
 - [:graph:] 可见的字符, 不包括空白
 - [:lower:] 小写字母
 - [:print:] 可见的字符和空白
 - [:punct:] 标点
 - [:space:] 空白符, 包括 \t \r \n
 - \s 与 [[[:space:]]] 同义, \S 与 [^[:space:]] 同义
 - [:upper:] 大写字母
 - [:xdigit:] 十六进制数

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配

练习

流编辑器 sed

bash 正则表

达式

Perl 正则表

达式

Python 正则表

达式

调试

常见问题

```
grep tex usr_share_doc_list.txt | head
```

```
drwxr-xr-x  4 root root 4096 Feb  8  2021 auctex
drwxr-xr-x  3 root root 4096 Feb  8  2021 chktex
drwxr-xr-x  4 root root 4096 Feb  8  2021 dot2tex
drwxr-xr-x  2 root root 4096 Feb  8  2021 fonts-texgyre
drwxr-xr-x  2 root root 4096 Feb  8  2021 gettext
drwxr-xr-x  2 root root 4096 Feb  8  2021 gettext-base
drwxr-xr-x  2 root root 4096 Feb  8  2021 latex2rtf
lrwxrwxrwx  1 root root   16 May 11  2016 latex-cjk-chinese -> latex-cjk-common
drwxr-xr-x  5 root root 4096 Apr 14 15:46 latex-cjk-common
drwxr-xr-x  3 root root 4096 Aug  8  2017 latex-mk
```

- 去掉 text texgyre ['dʒaɪə, 'gɪə] texinfo

```
egrep 'tex([^tgi]|$)' usr_share_doc_list.txt | head
```

```
drwxr-xr-x  4 root root 4096 Feb  8  2021 auctex
drwxr-xr-x  3 root root 4096 Feb  8  2021 chktex
drwxr-xr-x  4 root root 4096 Feb  8  2021 dot2tex
drwxr-xr-x  2 root root 4096 Feb  8  2021 latex2rtf
lrwxrwxrwx  1 root root   16 May 11  2016 latex-cjk-chinese -> latex-cjk-common
drwxr-xr-x  5 root root 4096 Apr 14 15:46 latex-cjk-common
drwxr-xr-x  3 root root 4096 Aug  8  2017 latex-mk
drwxr-xr-x  3 root root 4096 Feb  8  2021 latexmk
drwxr-xr-x  2 root root 4096 Feb  8  2021 libpod-latex-perl
drwxr-xr-x  2 root root 4096 Apr 14 15:44 libptexenc1
```



```
egrep 'tex\S*latex' usr_share_doc_list.txt | head
```

```
drwxr-xr-x  2 root root 4096 Apr 14 15:45 texlive-latex-base
drwxr-xr-x  2 root root 4096 Apr 14 15:45 texlive-latex-extra
drwxr-xr-x  2 root root 4096 Apr 14 15:45 texlive-latex-recommended
```

```
egrep '\btex' usr_share_doc_list.txt | head
```

```
drwxr-xr-x 2 root root 4096 Feb  8 2021 fonts-texgyre
drwxr-xr-x 2 root root 4096 Feb  8 2021 libdjvulibre-text
drwxr-xr-x 2 root root 4096 Apr 14 15:44 tex-common
drwxr-xr-x 2 root root 4096 Mar 24 11:19 tex-gyre
drwxr-xr-x 3 root root 4096 Feb  8 2021 texinfo
drwxr-xr-x 4 root root 4096 Apr 14 15:46 texlive-base
drwxr-xr-x 2 root root 4096 Apr 14 15:45 texlive-bibtex-extra
drwxr-xr-x 2 root root 4096 Apr 14 15:44 texlive-binaries
drwxr-xr-x 25 root root 4096 Feb  8 2021 texlive-doc
drwxr-xr-x 2 root root 4096 Apr 14 15:46 texlive-extra-utils
```

```
egrep '\<python3\>' usr_share_doc_list.txt | head
```

```
drwxr-xr-x  2 root root 4096 Apr 14 15:44 python3
drwxr-xr-x  2 root root 4096 Apr 14 15:44 python3.9
drwxr-xr-x  2 root root 4096 Apr 14 15:44 python3.9-minimal
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-apt
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-attr
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-automat
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-backcall
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-bcrypt
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-cairo
drwxr-xr-x  2 root root 4096 Feb  8 2021 python3-certifi
/bin/grep: write error: Broken pipe
```

```
egrep '\b[[:alnum:]]{3}\b' usr_share_doc_list.txt | head
```

```
drwxr-xr-x 2 root root 4096 Aug 8 2017 abootimg
drwxr-xr-x 2 root root 4096 Feb 8 2021 acl
drwxr-xr-x 2 root root 4096 Jul 2 2019 acpi
drwxr-xr-x 2 root root 4096 Feb 8 2021 adb
drwxr-xr-x 3 root root 4096 Oct 12 2019 adduser
drwxr-xr-x 2 root root 4096 Feb 8 2021 adwaita-icon-theme
drwxr-xr-x 2 root root 4096 Jul 2 2019 alsa-oss
drwxr-xr-x 6 root root 4096 Feb 8 2021 alsa-tools
drwxr-xr-x 8 root root 4096 Feb 8 2021 alsa-tools-gui
drwxr-xr-x 2 root root 4096 Feb 8 2021 alsa-utils
/bin/grep: write error: Broken pipe
```

- sed 是 stream editor，在“流”上进行编辑
- 常用的操作是替换，可以使用正则表达式
 - 支持 Basic RE 和 Extended RE -r

文档

- man sed
- info sed
- 名字是 streamed ed，前身是 ed

替换

```
echo hello | sed 's/lo/do/'
```

heldo

- 把 9 处的 “,” 转为换行
- g 代表全部替换

```
seq -s, -w 00 99 | sed 's/9,/9\n/g'
```

```
00,01,02,03,04,05,06,07,08,09
10,11,12,13,14,15,16,17,18,19
20,21,22,23,24,25,26,27,28,29
30,31,32,33,34,35,36,37,38,39
40,41,42,43,44,45,46,47,48,49
50,51,52,53,54,55,56,57,58,59
60,61,62,63,64,65,66,67,68,69
70,71,72,73,74,75,76,77,78,79
80,81,82,83,84,85,86,87,88,89
90,91,92,93,94,95,96,97,98,99
```

- 在 4 或 9 处换行
- \1 代表第一个括号中的成分

```
seq -s, -w 00 99 | sed -r 's/([49]),/\1\n/g'
```

00,01,02,03,04

05,06,07,08,09

10,11,12,13,14

15,16,17,18,19

20,21,22,23,24

25,26,27,28,29

30,31,32,33,34

35,36,37,38,39

40,41,42,43,44

45,46,47,48,49

50,51,52,53,54

55,56,57,58,59

60,61,62,63,64

65,66,67,68,69

70,71,72,73,74

75,76,77,78,79

80,81,82,83,84

85,86,87,88,89

90,91,92,93,94

95,96,97,98,99

- 多个替换可以用 -e 分开 jammy ['dʒami]

```
sed -e 's,archive.canonical.com/,mirrors.tuna.tsinghua.edu.cn/, ' \
    -e 's/trusty/jammy/' sources.list | head
```

```
# deb cdrom:[Ubuntu 14.04 LTS _Trusty Tahr_ - Release amd64 (20140417)]/ jammy main
```

```
# See http://help.ubuntu.com/community/UpgradeNotes for how to upgrade to
# newer versions of the distribution.
```

```
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe mult
```

```
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe
```

```
## Major bug fix updates produced after the final release of the
## distribution.
```

```
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted unive
```


- 多个替换也可以用 ; 分隔
- -n 默认不输出, p 来输出

```
sed -n '/^[^#]/{s,archive.canonical.com/,mirrors.tuna.tsinghua.edu.cn/,;  
s/trusty/jammy/;p}' sources.list | head
```

```
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe mult  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe  
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted unive  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted u  
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted uni  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted  
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted univ  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted
```

- 使用匹配引用 \1

```
sed -nr '/^[^#]/{s,archive.canonical.com(.*)trusty,mirrors.tuna.tsinghua.edu.cn\1ja  
p}' sources.list | head
```

```
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe mult  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe  
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted unive  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted u  
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted uni  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted  
deb http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted univ  
deb-src http://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted
```

```
egrep -v '^(#.*)?$' sources.list
```

```
deb http://archive.canonical.com/ubuntu/ trusty main restricted universe multiverse
deb-src http://archive.canonical.com/ubuntu/ trusty main restricted universe multiv
deb http://archive.canonical.com/ubuntu/ trusty-updates main restricted universe mu
deb-src http://archive.canonical.com/ubuntu/ trusty-updates main restricted univers
deb http://archive.canonical.com/ubuntu/ trusty-backports main restricted universe
deb-src http://archive.canonical.com/ubuntu/ trusty-backports main restricted unive
deb http://archive.canonical.com/ubuntu/ trusty-security main restricted universe m
deb-src http://archive.canonical.com/ubuntu/ trusty-security main restricted univer
```

- 达成成就：没有任何字母数字的正则表达式

- 由 `[[ EXPR =~ REGEX ]]` 判断 EXPR 是否能被 REGEX 匹配。
 - 支持扩展正则表达式
 - 匹配结果由 BASH_REMATCH 列表给出
- 比每次调用 egrep 快很多

```
seq 1000 | until ! read line
do
    if [[ $line =~ ^23+$ ]]; then
        echo ${line}
        echo ${BASH_REMATCH[0]}
    fi
done
```

```
23
23
233
233
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配

练习

流编辑器 sed

bash 正则表达式

Perl 正则表达式

Python 正则表达式

调试

常见问题

```
while read entry; do
    if [[ $entry =~ ^([^\#].*)archive.canonical.com(.*?)trusty ]]; then
        echo ${BASH_REMATCH[1]} ${BASH_REMATCH[2]}
    fi
done < sources.list
```

```
deb http:// /ubuntu/
deb-src http:// /ubuntu/
deb http:// /ubuntu/
deb-src http:// /ubuntu/
deb http:// /ubuntu/
deb-src http:// /ubuntu/
deb http:// /ubuntu/
deb-src http:// /ubuntu/
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配
练习

流编辑器 sed

bash 正则表
达式Perl 正则表
达式Python 正则表
达式

调试

常见问题

OSIRIS 的文件整理。

```
/eos/juno/Commissioning_Dryrun/OSIRIS/2024/0311/OSIRISData_hybrid_20240311_101439.c
```

链接成

```
/junofs/OSIRIS/0311/101439.h5
```

```
pattern='\.' # 引号被 bash 处理
```

```
[[ . =~ $pattern ]] # 成功
```

```
[[ . =~ \. ]] # 成功
```

```
[[ . =~ "$pattern" ]] # 失败
```

```
[[ . =~ '\.' ]] # 失败
```

- \ 同时在 bash 和正则语句意为“除去特殊含义”。需要格外注意
- 引号在变量赋值小时使用时，变量的值不含引号。
 - 在 [[]] 里写引号时，所有字符都没有特殊含义

- grep 的多行匹配功能很弱，应当使用 pcre2grep
- pcre2 吸纳了 Perl 的正则表达式，一部分来自 Python 正则表达式。

```
columns=( "DATASET.*SER"
          "DATATYPE.*F64"
          "DATASET.*SER_stddev"
          "DATATYPE.*F64"
        )

h5dump -A ser.h5| pcre2grep -M "$(echo ${columns[@]} | sed 's/ /(.\|\\n)*\/g')"
echo $?
```

```
DATASET "SER" {
    DATATYPE  H5T_IEEE_F64LE
    DATASPACE  SIMPLE { ( 100 ) / ( 100 ) }
}
DATASET "SER_stddev" {
    DATATYPE  H5T_IEEE_F64LE
```

0

- 基本正则表达式与有限自动机 finite automata 等价
- 在 FA 之外时，可引入更强大的处理功能

提供类似 sed 的命令语法

- Perl 的 -p 选项
causes Perl to assume the following loop around your program, which makes it iterate over filename arguments somewhat like sed:

```
while (<>) {  
    ...                # your program goes here  
} continue {  
    print or die "-p destination: $!\n";  
}
```

- 前趋引用匹配：形似 sed 的替换
- iterative matches \G where previous match ends
- look ahead behind
- atomic grouping 不再缩短

选择结构

- conditional constructs (140)
 - 使用之前的匹配 ($<$)?\w+(?(1)>)

递归结构

- 由 pcre 提出，用于匹配形如 $a^n b^n$ 的平衡问题。

例子：使用“回头看”构造

- 如果逗号之前为 4 或 9，就把它替换成换行。

```
seq -s, -w 00 49 | perl -p -e 's/(?<=[49]),/\n/g'
```

```
00,01,02,03,04
05,06,07,08,09
10,11,12,13,14
15,16,17,18,19
20,21,22,23,24
25,26,27,28,29
30,31,32,33,34
35,36,37,38,39
40,41,42,43,44
45,46,47,48,49
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配
练习

流编辑器 sed

bash 正则表
达式Perl 正则表
达式Python 正则表
达式

调试

常见问题

受 Perl 正则表达的影响，语法参数了 Perl 正则表达式。

```
import re  
  
m = re.search('(?<=abc)def', 'abcdef')  
m.group(0)  
  
def
```

正则表达式

续本达

复习与提示

正则表达式

应用实例

文件名匹配
练习

流编辑器 sed

bash 正则表
达式

Perl 正则表
达式

Python 正则表
达式

调试

常见问题

- Perl 的正则表达式调试和教学工具

- greedy: 贪婪匹配
- lazy: 最小匹配

	?	+	*
greedy	?	+	*
lazy	??	+?	*?